



UNIVERSITY OF
CAMBRIDGE

sth like Mechanistic Interpretability of Large Language Models

Elisabeta-Iulia (Julia) Dima



King's College

This dissertation is submitted on June, 2026 for the degree of Master of Philosophy

Abstract

My abstract ...

Declaration

This dissertation is the result of my own work and includes nothing which is the outcome of work done in collaboration except as declared in the Preface and specified in the text. It is not substantially the same as any that I have submitted, or am concurrently submitting, for a degree or diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the Preface and specified in the text. I further state that no substantial part of my dissertation has already been submitted, or is being concurrently submitted, for any such degree, diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the Preface and specified in the text. This dissertation does not exceed the prescribed limit of 60 000 words.

Elisabeta-Iulia (Julia) Dima

June, 2026

Acknowledgements

My acknowledgements ...

Contents

1	Introduction	11
1.1	Mechanistic Interpretability	11
2	Related Work	13
2.1	Arithmetic algorithms in transformers	13
3	Concept Localisation and Intervention	15
3.1	Token Predicate Abstraction	15
3.2	The Soft FSM Router	17
3.2.1	Architecture	17
3.2.2	Supervised Training	18
3.2.3	Predicate Selectivity and Natural-Language Robustness	19
3.3	Steering Vector Extraction	20
3.3.1	Anchor Position	20
3.3.2	Contrastive Extraction	20
3.4	Local Steering at Inference Time	21
3.4.1	Evaluation Protocol	22
3.5	Analysis of Extracted Steering Vectors	22
4	Concept Localisation via Contrastive Residual-Stream Analysis	25
4.1	Contrastive Pair Construction	26
4.2	Layer-Wise Concept Deltas	27
4.3	Sharpness and Concept Localisation	27
4.4	Feature-Space Interpretation	28
4.5	Template Robustness	29
4.6	Broader Concept Suite	29
4.7	Results	30
4.7.1	Norm Trajectory and Phase Structure	30
4.7.2	Representational Geometry: Cross-Layer Similarity	32
4.7.3	Template Robustness and Output Preparation	33
4.7.4	Feature-Space Decomposition	33

4.8 Relation to Steering Effectiveness	35
Bibliography	39
Glossary	41

Chapter 1

Introduction

Understanding what neural networks actually *compute*, rather than what they *output*, is the central problem of mechanistic interpretability. A model that correctly solves arithmetic problems or chains reasoning steps coherently need not be implementing anything resembling the algorithm a human would use. Whether it does, where in the network that computation lives, and whether it can be manipulated predictably are empirical questions that evaluation benchmarks cannot answer.

1.1 Mechanistic Interpretability

Anthropic’s *On the Biology of a Large Language Model* [1] proposes a reproducible methodology for approaching such questions on a large language model (LLM). Combining sparse autoencoders (SAEs) with cross-layer transcoders (CLTs) and backward-pass attribution, it constructs dependency graphs that trace information flow from input tokens through intermediate features to output logits. The framework is the starting point of mechanistic interpretability work done in this thesis, focused on how mathematical primitives are implemented by LLMs, and how these implementations can be improved through insights from mechanistic interpretability.

Prior mechanistic work has established that small transformers trained on arithmetic tasks converge on specific and inspectable algorithms rather than fitting statistical regularities [3, 8, 5, 6, 2]. Whether analogous structure arises in a large instruction-tuned model, trained without an explicit arithmetic curriculum on substantially more diverse data, is considerably less clear and is a central concern of this work. Furthermore, since mechanistic interpretability offers relevant insights into how these models perform algorithmic tasks, we can make use of this knowledge to improve model performance on algorithmic tasks, which is the focus of the second part of this thesis.

We take the open access model `Qwen3` as our subject model and integer addition as our primary task. Addition is a reasonable prototype due to having unambiguous

ground truth and being structured enough that prior work has identified non-trivial internal mechanisms [3, 5], and performance degrades in interpretable ways as digit magnitude increases. We initially characterise the model’s behaviour on addition using the existing attribution graph methodology given by Lindsey et al. [1]. Beyond reproducing the attribution graphs, we perform intervention studies such as constrained patching to validate hypotheses regarding the underlying computation. We also use linear probes to localise where arithmetic-relevant information is encoded across layers.

By drawing insights from our mechanistic analysis, we then proceed to improve model performance on mathematical primitives through modifying specific components of the model.

The rest of this dissertation is structured as follows. Chapter 2 covers the relevant background.

Chapter 2

Related Work

2.1 Arithmetic algorithms in transformers

A productive line of mechanistic interpretability research has asked not merely whether transformers can perform arithmetic, but *how* they do so: what internal representations they form and what computational steps those representations support.

Nanda et al. [3] studied a one-layer transformer trained on modular addition and found that grokking, the phenomenon whereby generalisation appears suddenly after extended training on memorised data, corresponds to the emergence of a Fourier-based algorithm. The trained model encodes operands as superpositions of trigonometric features and composes them across layers to compute the result. The emergence of this structure can be tracked through interpretable progress measures even before the accuracy curve improves significantly.

Zhong et al. [8] revisited modular addition and showed that the same task admits two qualitatively distinct algorithmic solutions: a Fourier-based strategy they term the “clock” algorithm, and a lookup-table strategy they term the “pizza” algorithm. Which solution emerges depends on training configuration, demonstrating that identical supervision can produce structurally different circuits and that mechanistic diversity is not solely an artefact of random initialisation.

For standard integer addition, Quirke and Barez [5] identified a carry-propagation circuit distributed across attention and MLP layers, bearing a structural resemblance to the long-addition algorithm familiar from elementary arithmetic. Quirke et al. [6] extended this account to subtraction, characterising how the same underlying circuit adapts to accommodate sign.

Musat [2] approached the problem from the perspective of training dynamics, tracing how the internal representations of operands and results cluster and align over the course of training on modular addition. Their analysis shows that organised algorithmic structure is not installed gradually but emerges through identifiable phase transitions, reinforcing

the view that grokking marks a qualitative change in the model’s internal organisation rather than a smooth accumulation of generalising evidence.

These results establish a strong prior that transformers trained specifically on arithmetic tasks do not merely interpolate over training examples but converge on discrete, inspectable computational procedures. All of the models studied, however, are small transformers trained from scratch on a single arithmetic task. The present work asks whether comparable structure is recoverable in **Qwen3**, a multi-billion-parameter instruction-tuned model trained on a broad natural-language corpus with no explicit arithmetic curriculum.

Chapter 3

Concept Localisation and Intervention

Mechanistic analysis of Qwen3-4B on integer arithmetic, following the methodology of Lindsey et al. [1], reveals that the residual stream at the position corresponding to the completed arithmetic expression contains information relevant to downstream answer generation. Prior work has shown that simple linear probes trained on hidden representations can accurately decode both arithmetic outputs and model correctness from internal activations, indicating that these representations are often linearly accessible [?]. This observation motivates a concrete intervention. The mean difference in hidden state between prompts on which the model succeeds and those on which it fails defines a direction in representation space that, when added at inference time, should reinforce correct computation. To act on this hypothesis precisely, we require a mechanism that identifies *where* within a prompt a given primitive appears and with what confidence, rather than applying a uniform perturbation to every token. We therefore design a *PrimitiveRouter*, a set of four learnable soft finite-state machines trained jointly on synthetic arithmetic expressions to detect which primitive operation is active at each token position. The router converges to perfect accuracy on its held-out validation set within 30 epochs, providing a reliable positional signal. Its output gates the injection of contrastive steering vectors extracted from the model’s own activations, producing a closed-loop intervention that requires no modification of model weights and that handles prompts containing multiple primitives simultaneously. Figure 3.1 gives an overview of the complete pipeline.

3.1 Token Predicate Abstraction

Arithmetic expressions share structural patterns that are independent of the specific operand values they contain. The numeral thirty-seven and the numeral one thousand play structurally identical roles in their respective expressions, and what varies is only

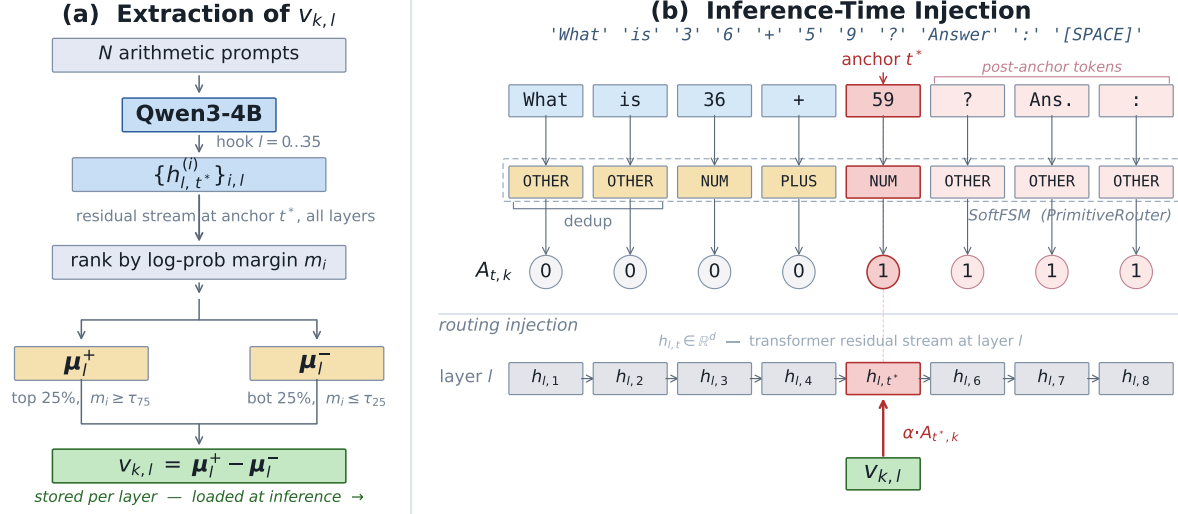


Figure 3.1: Overview of FSM routing and local steering on the prompt **What is 36+59? Answer: .** Tokens are mapped rule-based to predicates; consecutive identical predicates are collapsed (dedup) so that multi-digit numerals resolve to a single **NUM** step regardless of how many tokens the tokeniser produces. The SoftFSM processes the predicate sequence and outputs per-position activation values $A_{t,k}$ for the target primitive k . The position where the complete pattern first fires (token 9, the second operand) is identified as the anchor t^* ; the prompt continues with three further tokens (**?**, **Answer**, **:**) after the anchor, illustrating that the anchor need not coincide with the final token of the prompt. At inference time the steering vector $v_{k,l}$ is injected into the residual stream of Qwen3-4B at layer l and position t^* , scaled by $\alpha \cdot A_{t^*,k}$. The final space token is added to encourage the model to generate the answer token, instead of closing punctuation marks.

the surface form. We exploit this by defining a shared predicate vocabulary of eleven categories (Table 3.1), each mapping a class of surface tokens to a structural role. The mapping is deterministic and rule-based, requiring no learned components. Multi-digit integers, regardless of their magnitude, all map to **NUMBER**; operator symbols and their written equivalents map to the corresponding operator predicate; unrecognised tokens map to **OTHER**.

Qwen3 tokenises multi-digit integers digit-by-digit, so the string 37 produces two separate tokens. To ensure that the predicate sequence seen by the router is independent of this tokenisation granularity, consecutive tokens that map to the same predicate are collapsed into a single group. The token sequence ["3", "7", "+", "1", "1", "5"] therefore produces the predicate sequence [NUMBER, PLUS, NUMBER], which is structurally identical to the sequence produced by a word-level tokeniser on the same expression.

Table 3.1: Shared predicate vocabulary. Each predicate covers all surface forms listed; the mapping is value-independent.

Predicate	Surface forms
NUMBER	Any integer or decimal literal
PLUS	+, plus, add
MINUS	-, minus, subtract
TIMES	*, ×, ·, times, mul, multiply
DIV	/, ÷, div, divide
MOD	%, mod, modulo
EQUALS	=, ==, equals, is
OPEN_PAREN	(, [, {
CLOSE_PAREN	),], }
COMMA	,
OTHER	Any unrecognised token

3.2 The Soft FSM Router

3.2.1 Architecture

We model each arithmetic primitive as a soft, learnable finite-state machine (SoftFSM) operating on the predicate sequence. Rather than maintaining a one-hot state indicator, the machine tracks a probability distribution over S latent states, which allows gradients to flow through transitions and makes training by standard gradient descent possible. At each predicate step t , the state is updated by

$$s_{t+1} = \text{softmax}(s_t T_{p_t}), \quad (3.1)$$

where $s_t \in \mathbb{R}^S$ is the current state vector (with $s_0 = e_1$, all probability mass in state 0), and $T_{p_t} \in \mathbb{R}^{S \times S}$ is the learned transition matrix for predicate p_t . A scalar readout at each step produces the activation value for primitive k ,

$$A_{t,k} = \sigma(w_k^\top s_t + b_k), \quad (3.2)$$

where $w_k \in \mathbb{R}^S$ and $b_k \in \mathbb{R}$ are learned readout parameters, and σ denotes the sigmoid function. The value $A_{t,k} \in [0, 1]$ therefore measures how confidently the router believes that primitive k is active at predicate step t .

A *PrimitiveRouter* module wraps $K = 4$ independent SoftFSM instances (addition, subtraction, multiplication, modular reduction) sharing the same predicate vocabulary but with separate transition matrices and readout parameters. The router produces $A \in \mathbb{R}^{B \times T \times K}$. Each SoftFSM uses $S = 4$ latent states, contributing $P \cdot S^2 + S + 1 = 11 \times 16 + 5 = 181$ parameters per primitive (176 for the transition matrices, 4 for the readout weights, and 1 for the readout bias), giving a total of $K \times 181 = 724$ parameters

across all four primitives, a negligible overhead relative to the main model.

Transition matrices are initialised with a strong identity bias, $T_p += 3I$, so that each machine begins as a “stay put” automaton and learns to deviate only on structurally relevant predicates. Readout biases are initialised to $b_k = -2$, keeping activations near zero at initialisation so that the router does not fire spuriously on arbitrary inputs.

3.2.2 Supervised Training

The router is trained on synthetically generated arithmetic expressions with rule-based per-token labels. For each example, the label $y_{t,k} \in \{0, 1\}$ is set to 1 at all positions after the complete pattern for primitive k has appeared in the expression, and to 0 before. This cumulative, once-set labelling reflects the router’s role as a detector of expression completion rather than a local classifier. For the expression `37 + 115`, the addition label becomes 1 at the second `NUMBER` token and remains 1 for all subsequent tokens, while all other primitive labels remain 0.

Training minimises binary cross-entropy averaged over all valid token positions and all primitives,

$$\mathcal{L} = -\frac{1}{|\mathcal{V}|} \sum_{(b,t) \in \mathcal{V}} \sum_{k=1}^K \left[y_{t,k} \log A_{t,k} + (1 - y_{t,k}) \log(1 - A_{t,k}) \right], \quad (3.3)$$

where $\mathcal{V}$ is the set of valid (batch, position) index pairs, excluding padding. The learning rate follows a cosine schedule with a linear warmup over the first 10% of training steps. Table 3.2 gives the complete training configuration. The router achieves 100% per-primitive binary accuracy on the held-out validation set across all four primitives by epoch 7 (approximately 750 gradient steps), and training continues to epoch 30 to minimise the cross-entropy further, reaching a final value of 0.0261.

Table 3.2: FSM router training configuration.

Hyperparameter	Value
Total expressions	8000
Training split	85% (6800)
Validation split	15% (1200)
Epochs	30
Batch size	64
Optimiser	AdamW
Learning rate	3×10^{-3}
Weight decay	10^{-4}
Gradient clip norm	1.0
LR schedule	Cosine with 10% linear warmup
Final loss	0.0261
Final primitive accuracy	100% (all four primitives)

The expression templates used during training span a range of surface forms for each operation (for example, $\{a\} + \{b\}$, $\{a\}$ plus $\{b\}$, compute $\{a\} + \{b\}$), as well as mixed expressions containing multiple primitives. Operands are drawn uniformly from $[1, 999]$. Because the input to the router is the predicate sequence rather than raw token IDs, the trained model generalises immediately to any operand magnitude.

3.2.3 Predicate Selectivity and Natural-Language Robustness

A potential concern for prompts that mix natural language with arithmetic is whether the router fires spuriously on incidental numeric tokens. The sentence *I have 3 apples and 5 oranges* contains two numeric tokens and the word *and*, which is not in the keyword map and therefore maps to **OTHER**. The resulting predicate sequence is **NUMBER OTHER OTHER NUMBER**, which never contains a **PLUS** predicate. Because the addition FSM is initialised with a strong identity bias and trained exclusively on expressions of the form **NUMBER PLUS NUMBER**, the **PLUS** predicate is the load-bearing gating condition: without it, the FSM remains in its intermediate state and the readout stays near zero throughout.

This property holds by construction of the predicate vocabulary in `predicates.py`. Only explicit operator symbols (+, *, %, and so on) and their written equivalents (*plus*, *times*, *mod*) are mapped to operator predicates. Words that are semantically suggestive of an operation but not part of the keyword map (*and*, *with*, *more than*) resolve to **OTHER** and are invisible to the FSM transitions.

We verify this empirically by running the trained router on several probe inputs.

Table 3.3: Router addition activation ($A_{t,\text{add}}$) at each token for representative probe inputs. The router fires only when an explicit operator predicate appears between two numeric tokens.

Input	Token at peak	Peak $A_{t,\text{add}}$
calc: 36+59=	59 (after +)	0.906
I have 3 apples and 5 oranges	none	0.014
I have 3 + 5 things	5 (after +)	0.906
3 things and 5 more	none	0.014

The router reaches 0.906 only when an explicit + symbol is present between two numeric tokens, regardless of surrounding natural-language context. Prompts containing two numbers with no operator predicate between them produce activations indistinguishable from the baseline near-zero level (0.014), confirming that the FSM learned the strict pattern **NUMBER PLUS NUMBER** rather than a looser co-occurrence of numerals. In a mixed prompt such as *The answer to 3 apples and 5 oranges is calc: 3+5=*, the router ignores the natural-language numbers entirely and fires only at the calculator expression.

3.3 Steering Vector Extraction

3.3.1 Anchor Position

For each input prompt, the FSM router maps the predicate sequence derived from the Qwen tokenisation to a per-position activation vector. The *anchor position* t^* is the last token index of the predicate group at which $A_{t,k}$ is maximised for the target primitive k , subject to $\max_t A_{t,k} \geq \tau_{\text{FSM}} = 0.5$. This position corresponds to the point at which the complete pattern has been recognised, and is therefore the natural site at which to extract a representation of the pending computation and also the site at which steering injection will be applied at inference time. Anchoring extraction and injection at the same position ensures that the injected vector lies in the same distributional neighbourhood as the activations from which it was computed. This consistency is necessary because residual-stream representations are strongly position-dependent: a vector extracted at one token position carries contextual information specific to that position, and injecting it at a different position would place it out of distribution with respect to the activations already present there, undermining the causal interpretation of the intervention.

3.3.2 Contrastive Extraction

For each primitive k , we generate $N = 3000$ single-primitive prompts of the form `calc: a OP b=` and run the model greedily to obtain logits for the first output token. Rather than using binary correctness as a bucketing signal, we compute a log-probability margin for each prompt,

$$m_i = \log P(y_1^* | x_i) - \max_{y \neq y_1^*} \log P(y | x_i), \quad (3.4)$$

where y_1^* is the correct first output token and x_i is the i -th prompt. A positive margin indicates that the model assigns higher probability to the correct token than to any alternative; a strongly negative margin indicates near-certain failure. This continuous signal is preferable to binary correctness for primitives on which baseline accuracy is low, because it allows all 3000 prompts to contribute to the extraction rather than only the small fraction on which the model happens to succeed.

Prompts are ranked by m_i . The top quartile ($m_i \geq \tau_{75}$, the 75th percentile) forms the high-confidence set and the bottom quartile ($m_i \leq \tau_{25}$, the 25th percentile) forms the low-confidence set; the middle 50% is discarded to maximise the contrast between the two groups. For each layer $l \in \{0, \dots, 35\}$ and each primitive k , residual stream activations at the anchor token t^* are collected from up to $n_{\text{bucket}} = 200$ prompts in each set. The

per-layer steering vector is their mean difference,

$$v_{k,l} = \mathbb{E}[h_{l,t^*} \mid m_i \geq \tau_{75}] - \mathbb{E}[h_{l,t^*} \mid m_i \leq \tau_{25}]. \quad (3.5)$$

Steering vectors are extracted independently for all 36 transformer layers of **Qwen3-4B**, leaving the identification of the most effective layer to a subsequent evaluation sweep. The operand range for addition is drawn from $[1, 999]$, spanning one-, two- and three-digit numbers, to prevent the extracted direction from encoding the magnitude regime of the operands rather than the operation type. Table 3.4 summarises the full extraction configuration.

Table 3.4: Steering vector extraction configuration. The log-probability margin (Eq. 3.4) is used as the bucketing signal for all primitives.

Parameter	Value
Prompts per primitive	3000
Bucketing metric	Log-probability margin
High-confidence threshold	75th percentile of m_i
Low-confidence threshold	25th percentile of m_i
Max samples per bucket	200
FSM firing threshold	0.5
Layers extracted	0–35 (all 36)
Model precision	bfloat16
<i>Operand ranges by primitive</i>	
Addition	$a, b \in [1, 999]$
Subtraction	$a, b \in [100, 999], a \geq b$
Multiplication	$a, b \in [2, 99]$
Modular	$a \in [100, 999], b \in [2, 13]$

3.4 Local Steering at Inference Time

At inference time, the router is run on the tokenised prompt to produce an activation matrix $A_{\text{tok}} \in \mathbb{R}^{T \times K}$, where each token inherits the router score of its enclosing predicate group. Forward hooks registered at each selected layer l modify the residual stream by

$$h_{l,t} \leftarrow h_{l,t} + \alpha \sum_{k=1}^K A_{\text{tok},t,k} \cdot v_{k,l}, \quad (3.6)$$

where $\alpha > 0$ is a scalar amplitude and the sum is over all K active primitives. The gate $A_{\text{tok},t,k} \in [0, 1]$ ensures that the magnitude of the perturbation at position t is proportional to the router’s confidence that primitive k is active there. Tokens inside the recognised expression receive the steering signal scaled by the activation; tokens outside it receive negligible contributions when the router fires selectively. Only prompt tokens

are perturbed, as tokens generated autoregressively were not present when the predicate sequence was computed and therefore receive no injection. This design allows multiple arithmetic primitives to coexist within a single prompt without requiring an explicit segmentation, since each primitive’s vector is gated independently by the corresponding SoftFSM output.

A *global* ablation mode applies each $v_{k,l}$ uniformly at all token positions and with all primitives summed, without any position-dependent gating. Comparing the local and global modes isolates the contribution of the router’s positional selectivity.

3.4.1 Evaluation Protocol

The evaluation sweeps over all layers for which steering vectors have been extracted and over amplitudes $\alpha \in \{0.5, 1.0, 2.0, 5.0\}$ on a held-out set of 300 prompts per primitive. Three metrics are computed relative to an un-steered baseline. *First-token accuracy* measures whether the first generated token matches the correct answer. *Full-answer accuracy* measures whether the entire generated string is correct. *Digit accuracy* measures the fraction of digit characters in the generated string that match the corresponding character in the ground-truth answer, providing a more sensitive signal for multi-digit outputs where a model that recovers the leading digit but fails on later ones scores zero on full-answer accuracy but non-zero on digit accuracy.

An optional out-of-distribution evaluation resamples operands from a range not used during extraction (for example, four-digit operands in $[1000, 9999]$), providing a direct test of whether the steering direction transfers across magnitude regimes. The baseline and steered evaluations are conducted identically, with all metrics reported as absolute values and signed deltas relative to the unsteered model.

3.5 Analysis of Extracted Steering Vectors

Before evaluating the causal effect of steering on model accuracy, we examine the norms $\|v_{k,l}\|$ of the extracted vectors across all 36 layers and all four primitives. The norm at layer l for primitive k measures the Euclidean separation between the high- and low-confidence residual-stream clusters at that depth. Figure 3.2 plots this quantity at every layer.

The norms grow monotonically with depth for every primitive, but this trajectory is primarily a scale artefact. Residual-stream vectors accumulate magnitude through skip connections at every layer. As a result, $\|v_{k,l}\|$ at a given depth reflects the geometric scale of activations at that point rather than the informativeness of the corresponding direction. A large norm at layer 35 does not imply that layer 35 is the most effective injection site.

More informative than the overall trend is a consistent phase transition at layers 6 through 8 appearing across all four primitives. Below layer 6 norms remain small and

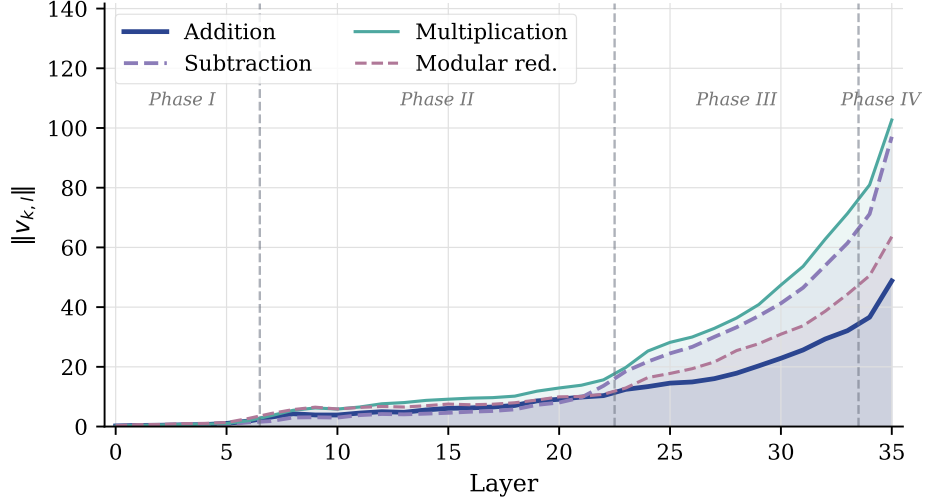


Figure 3.2: Steering vector norms $\|v_{k,l}\|$ at every layer for all four primitives. Phase boundaries (dashed lines) are placed at layers 5–6, 19–20, and 33–34, derived from the norm trajectories of these primitives. Phase I (layers 0–5): slow initial encoding. Phase II (layers 6–19): active separation regime, beginning with a rapid norm increase at layers 6–8 followed by moderate growth. Phase III (layers 20–33): accelerating scale accumulation with increasing inter-primitive divergence. Phase IV (layers 34–35): sharp final uptick visible across all primitives.

grow slowly; between layers 6 and 8 they increase by a factor of two to three over just two consecutive layers. This abrupt separation suggests that layers 6 through 8 are the earliest depth at which computation-mode information becomes linearly separable. The finding aligns with prior mechanistic analyses of small arithmetic transformers, which consistently locate algorithmic processing in early-to-middle layers [3, 5].

Immediately following this transition, a mild but consistent local dip appears at layers 9 and 10 across all four primitives. The norm retreats slightly before resuming its upward trend. This pattern is too consistent across primitives to be coincidental, and it plausibly marks the point at which one functional circuit completes its contribution and transfers control to the next.

Taken together, the observations suggest that computation-mode information is most distinctively encoded in roughly layers 8 through 20. Below this range the phase transition has not yet separated the clusters; above it, scale inflation begins to dominate absolute norms. This interval is the most natural target for steering, and the evaluation sweep confirms which specific layer and amplitude yields the largest accuracy gain.

Chapter 4

Concept Localisation via Contrastive Residual-Stream Analysis

The extraction procedure described in Chapter ?? partitions prompts by their log-probability margin. This signal reflects aggregate computational confidence rather than the state of any specific sub-operation within the arithmetic pipeline. The resulting steering vectors capture the mean difference between high- and low-confidence residual-stream distributions without isolating the contribution of individual sub-computations such as carry. Whether carry is encoded as a distinct, localised direction, and at what network depth this encoding first emerges, requires a more controlled experimental design than the confidence-based bucketing can provide.

This chapter investigates that question through a contrastive analysis in which prompt pairs are matched so that the only systematically varying factor is whether a units-column carry occurs. The methodology draws on representation engineering [9] and on contrastive activation addition [7], both of which extract concept directions by taking mean differences in activation space across semantically contrasting inputs. Here the technique serves as a diagnostic for concept localisation, using the layer-wise norm profile of the resulting delta vectors to characterise where the carry representation crystallises.

Carry is a natural target for this analysis because it is a binary, locally determined event. Given the units digits of both operands, whether a carry propagates is fixed. Quirke and Barez [5] identified a dedicated carry-propagation circuit in a small transformer trained on integer addition, establishing that carry can be implemented as a localised, mechanistically distinct sub-computation. Whether an analogous structure exists in Qwen3-4B is an open question. A large instruction-tuned model receives no explicit arithmetic curriculum, and finding a localised carry representation would suggest that mechanistic modularity is a general organisational principle rather than an artefact of task-specific training.

4.1 Contrastive Pair Construction

A carry pair consists of two prompts sharing the same first operand a and all but the units digit of the second operand b . Letting $a_0 = a \bmod 10$, the carry variant uses b^+ with $a_0 + (b^+ \bmod 10) \geq 10$, while the no-carry variant uses b^- with $a_0 + (b^- \bmod 10) < 10$. The two prompts produce token sequences of equal length differing at exactly one position, the units digit of b .

To confine the signal to a single carry event and exclude cascading carries into the tens column, we impose the constraint

$$(\lfloor a/10 \rfloor \bmod 10) + (\lfloor b/10 \rfloor \bmod 10) \leq 8, \quad (4.1)$$

so that the tens-column sum, even after receiving a units carry of 1, remains strictly below ten. Under this constraint the carry and no-carry prompts differ in exactly one computational event. The mean difference in residual-stream activations across many such pairs is therefore attributable to that single event rather than to higher-order carry interactions.

The anchor position t^* is identified by comparing the two tokenisations and taking the last index at which they differ. Because **Qwen3** represents each decimal digit as a single token, this corresponds to the units digit of b without exception, requiring no auxiliary alignment step. It is the earliest position at which the model holds the units digits of both operands in context and can in principle determine whether a carry will occur.

Pairs are generated for three prompt templates, T0 (`calc: {a}+{b}=`), T1 (`calc: {a} + {b} =`), and T2 (`What is {a}+{b}? Answer:`). The templates differ in whitespace and surface phrasing. Repeating the extraction across all three provides a check on whether the extracted direction reflects the model’s carry representation or a pattern tied to one specific tokenisation. Table 4.1 summarises the configuration.

Table 4.1: Concept localisation experimental configuration.

Parameter	Value
Pairs per template	200
Templates	T0, T1, T2
Operand digits	3
No-cascade constraint	$\lfloor a/10 \rfloor \bmod 10 + \lfloor b/10 \rfloor \bmod 10 \leq 8$
Anchor detection	Last differing token position
Layers extracted	0–35 (all 36)
Model precision	bfloat16
Top features reported per layer	15

4.2 Layer-Wise Concept Deltas

Let $\mathcal{D}_+$ and $\mathcal{D}_-$ denote the sets of carry and no-carry prompts respectively. For each layer $l \in \{0, \dots, L-1\}$, the carry concept direction is defined as the mean difference of residual-stream vectors at the anchor position,

$$\delta_l = \mathbb{E}_{x \in \mathcal{D}_+}[h_{l,t^*}(x)] - \mathbb{E}_{x \in \mathcal{D}_-}[h_{l,t^*}(x)], \quad (4.2)$$

where $h_{l,t}(x) \in \mathbb{R}^d$ is the residual-stream vector at layer l and position t for prompt x , consistent with the notation of Chapter ?? . Under the linear representation hypothesis [4], δ_l approximates the axis along which carry is encoded at depth l , provided the concept is linearly separable and the matched-pair averaging suppresses nuisance variation. The construction in Section 4.1 is designed to make that nuisance small.

The relationship between δ_l and the steering vectors $v_{k,l}$ of Chapter ?? is instructive. Both are mean-difference vectors in the residual stream, but they partition the prompt distribution along different axes. The steering vectors $v_{k,l}$ separate prompts by model confidence, capturing the direction that discriminates successful from unsuccessful computation. The concept deltas δ_l separate prompts by the presence or absence of a specific sub-operation, independently of whether the model ultimately produces the correct answer. The former is operationally motivated; the latter is conceptually motivated. Only the second admits a localisation analysis that is not confounded by aggregate performance.

4.3 Sharpness and Concept Localisation

The sequence $\{\|\delta_l\|\}_{l=0}^{L-1}$ constitutes a norm trajectory tracking how the representational distance between carry and no-carry conditions evolves with depth. A steep rise at a single layer followed by a plateau suggests the carry concept crystallises there. Gradual growth throughout suggests a distributed encoding with no dominant localisation site.

To quantify localisation, we define the sharpness index as the fraction of total norm mass concentrated within a window of width three centred at the peak layer $l^* = \arg \max_l \|\delta_l\|$,

$$\psi = \frac{\sum_{l=\max(0, l^*-1)}^{\min(L-1, l^*+1)} \|\delta_l\|}{\sum_{l=0}^{L-1} \|\delta_l\|}. \quad (4.3)$$

A value of ψ close to one indicates tight localisation around a single layer. A value close to $3/L$ indicates that the norm is approximately uniformly distributed with depth. The index thus provides a single scalar summary of the degree to which carry is encoded at a

specific processing stage.

Complementary to the norm trajectory, the inter-layer alignment $\rho_l = \cos(\delta_l, \delta_{l+1})$ measures whether the concept direction rotates between consecutive layers. Consistently high alignment would indicate a single direction that merely scales with depth. A sharp drop at a particular transition marks a qualitative reorientation, suggesting the carry concept is being re-encoded in a different subspace at that depth.

4.4 Feature-Space Interpretation

In the dictionary-learning framework applied to transformer internals, a *feature* is a learned direction $\phi_f \in \mathbb{R}^d$ hypothesised to correspond to a single interpretable concept or computation, where d is the residual-stream width of the transformer. Because transformers represent more independent concepts than they have dimensions, they are believed to superpose many approximately orthogonal feature directions within the residual stream, with each activation being a sparse signed combination of active features [1]. A transcoder operationalises this for MLP sub-layers as follows: an encoder $W_l^{\text{enc}} \in \mathbb{R}^{F_l \times d}$ maps the pre-MLP residual-stream vector to a sparse bottleneck of feature activations, and a decoder $W_l^{\text{dec}} \in \mathbb{R}^{d \times F_l}$ reconstructs the MLP output as a sum of active feature effects. Each encoder row $(W_l^{\text{enc}})_f$ is therefore a *detection direction* in $\mathbb{R}^d$.

The delta vectors δ_l are dense in $\mathbb{R}^d$ and do not directly identify which model components encode carry at each layer. The transcoder architecture provides a more interpretable way to decompose them sparsely. Each transcoder at layer l learns an encoder weight matrix $W_l^{\text{enc}} \in \mathbb{R}^{F_l \times d}$, where F_l is the number of learned features and d the residual-stream dimension. The f -th row $(W_l^{\text{enc}})_f$ is the direction to which feature f responds most strongly.

The cosine alignment between the carry delta and each encoder direction,

$$c_{l,f} = \frac{(W_l^{\text{enc}})_f \cdot \delta_l}{\|(W_l^{\text{enc}})_f\| \cdot \|\delta_l\|}, \quad (4.4)$$

identifies the transcoder features most aligned with the concept delta at that layer. Features with $|c_{l,f}|$ close to one are the monosemantic components whose activation pattern best accounts for the carry versus no-carry difference. This projection connects the dense, direction-based view of representation engineering to the sparse, feature-based view of dictionary learning. The same concept direction can thus be understood both as a point in residual-stream space and as a weighted combination of interpretable transcoder features, bridging two perspectives that the broader interpretability programme has pursued in parallel.

4.5 Template Robustness

Per-template deltas $\delta_l^{(T)}$ are computed separately for each template $T \in \{T0, T1, T2\}$, and their pairwise cosine similarities are measured at each layer. This provides a direct check on whether the extracted direction is a stable property of the model’s carry representation. Consistently high cross-template alignment at the layers where $\|\delta_l\|$ peaks would confirm that the same direction emerges regardless of surface form. Divergence at early layers would indicate that template-specific variation persists until mid-network processing assimilates the input into a common representational format, providing additional evidence about the depth at which template-invariant concept encoding begins. This consistency check is the analogue, at the level of individual sub-operations, of the cross-template steering evaluation in Chapter ??.

4.6 Broader Concept Suite

The contrastive methodology described in the preceding sections is not specific to carry. To test whether concept localisation generalises beyond arithmetic sub-operations, the same pipeline was applied to six additional concepts drawn from two categories. The first category, modular arithmetic, consists of concepts that are binary and locally determined: given a fixed number of input tokens, the answer is fully specified. The second category, relational and physical reasoning, consists of concepts whose evaluation requires integrating information across a longer context or appealing to world knowledge.

Table 4.2 lists all seven concepts together with the positive and negative conditions used to construct contrastive pairs. In every case the two conditions differ at exactly one token position. The resulting delta vector is therefore attributable to the concept shown rather than to differences in token count, surface phrasing, or operand magnitude.

Table 4.2: Concept suite used in the localisation experiments. For each concept, one token position changes between the positive and negative prompt; all other positions are identical. The anchor is always the last token at which the two sequences diverge.

Concept	Positive condition	Negative condition
Carry (mod 10)	units-digit sum ≥ 10 (carry occurs)	units-digit sum < 10 (no carry)
GCD divisibility	$\gcd(a, 7) = 7$ (divisible)	$\gcd(a, 7) = 1$ (coprime)
Residue class	$a \equiv 1 \pmod{7}$	$a \equiv 6 \pmod{7}$
Transitive ordering	chain $a > b > c$ holds	chain broken ($c > b$)
Energy conservation	bounce height $<$ drop height (physical)	bounce height $>$ drop height (violation)
Causal direction	A causes B (correct direction)	B causes A (reversed)
Negation scope	$m \leq n$ (negated comparison true)	$m > n$ (negated comparison false)

The modular arithmetic concepts share a structural property with carry: the binary outcome is determined by a single arithmetic relation between small integers, and the

concept is expressible as a test against a fixed modulus. The relational concepts require a different kind of evaluation. Transitive ordering and negation scope involve logical consistency across two or three values; energy conservation involves a physical plausibility judgment; causal direction involves the assignment of asymmetric roles to two entities. These distinctions motivate asking whether the two groups produce qualitatively different norm trajectories and whether their representations crystallise at similar depths.

Figure 4.1 shows the layer-wise delta norm for all seven concepts on a shared axis. Modular arithmetic concepts are drawn as solid lines; relational concepts as dashed lines. The right panel normalises each curve by its maximum value so that the growth profiles are comparable across concepts with very different absolute magnitudes.

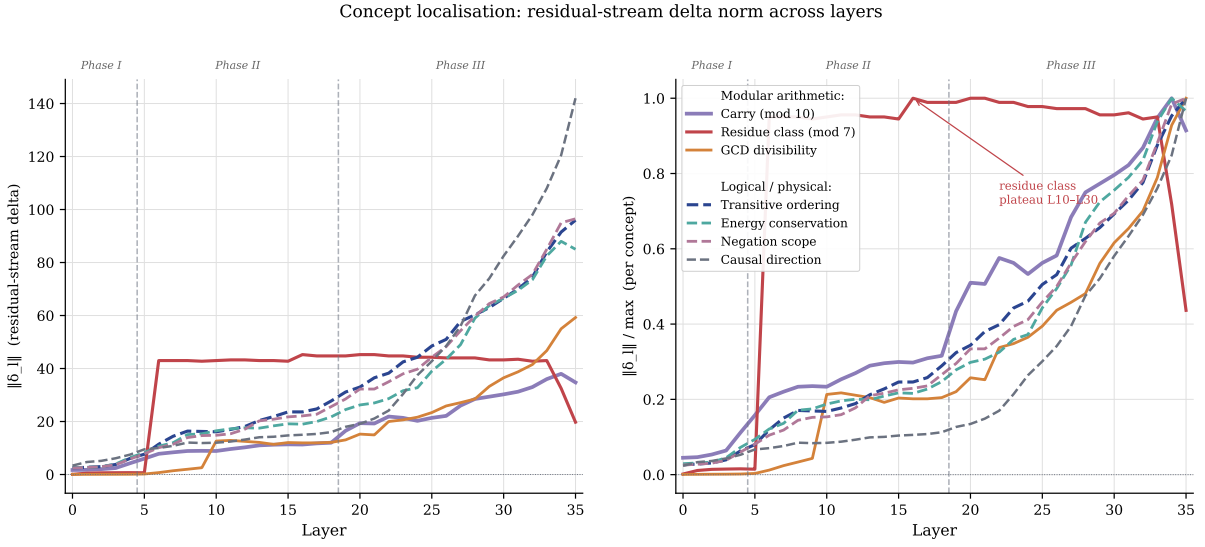


Figure 4.1: Layer-wise delta norm for all seven concepts. Left: raw norms. Right: norms divided by the per-concept maximum, so growth profiles are comparable. Modular arithmetic concepts (carry, GCD divisibility, residue class) are shown as solid lines; relational concepts (transitive ordering, energy conservation, causal direction, negation scope) as dashed lines. Vertical dashed lines mark the phase boundaries identified in the carry analysis.

4.7 Results

4.7.1 Norm Trajectory and Phase Structure

Figure 4.2 shows the layer-wise norm profile of the carry delta vector and the pairwise cross-template consistency on a shared axis, with three processing phases annotated. The norm trajectory reveals a clear three-stage structure that maps directly onto qualitatively distinct computational regimes.

In Phase I (layers 0–4), the norm rises slowly from 1.7 to 2.4 as the model processes

Carry concept localisation in Qwen3-4B

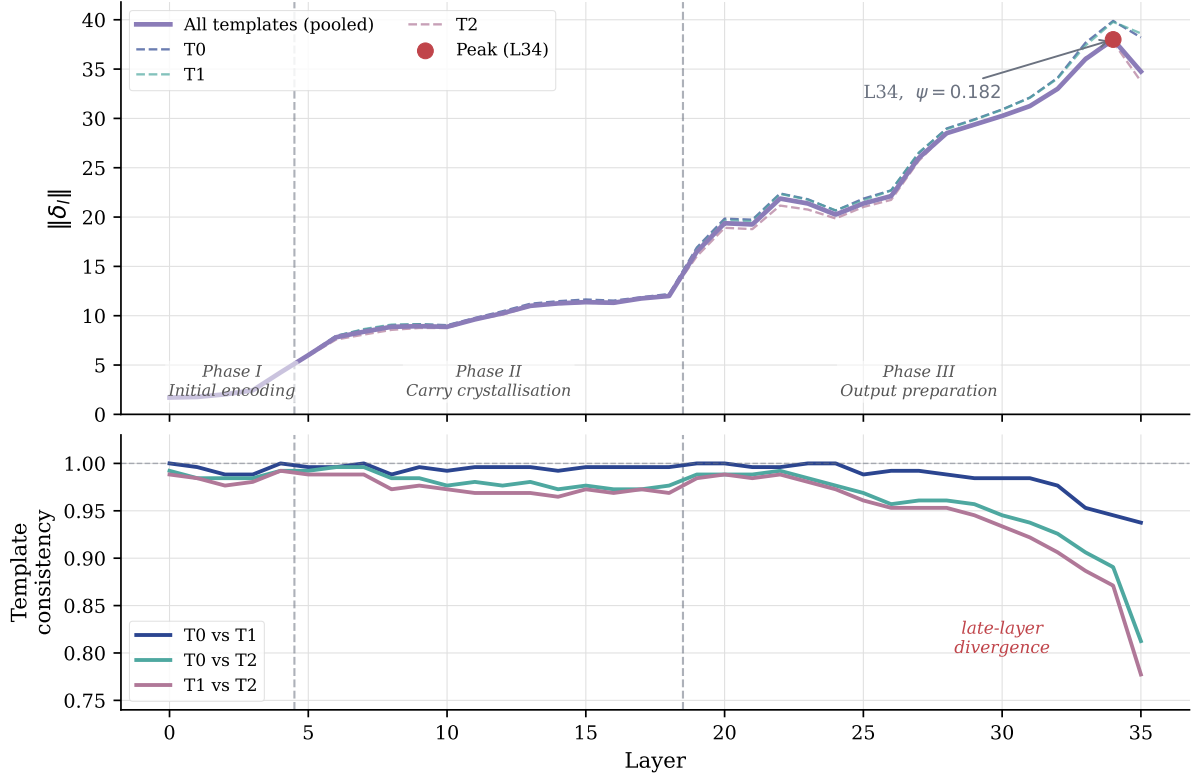


Figure 4.2: Top: norm of the carry concept delta $\|\delta_l\|$ with phase regions annotated and the peak at layer 34 marked. All four curves (pooled and per-template) are nearly indistinguishable, confirming template-invariant encoding throughout most of the network. Bottom: cross-template cosine similarity of the carry direction. The shaded red region marks the late-layer divergence zone where template-specific output preparation begins.

the raw token embeddings without yet forming a strong carry-specific representation. The first transition at layers 4–5 marks the entry into Phase II (layers 5–18), where the norm jumps to 6.0 and plateaus around 8–12: carry information crystallises here into a stable intermediate representation. A second transition at layer 18–19 raises the norm abruptly to 16.5, opening Phase III (layers 19–35), in which the representation grows steadily to its peak of 38.0 at layer 34 before dropping to 34.75 at the final layer. This terminal drop is consistent with the accumulated carry direction being partially projected into output-logit space at the last transformer block.

The sharpness index $\psi = 0.182$ quantifies the degree of localisation. For a 36-layer network, a uniform distribution would give $3/36 \approx 0.083$; the observed value is approximately twice this baseline, indicating that carry is a progressively enriched distributed representation rather than a sharply localised one, with growth concentrated at the two phase transitions.

4.7.2 Representational Geometry: Cross-Layer Similarity

To characterise how the carry direction evolves in representational geometry, Figure 4.3 shows the full 36×36 matrix of pairwise cosine similarities between delta vectors at all layer combinations.

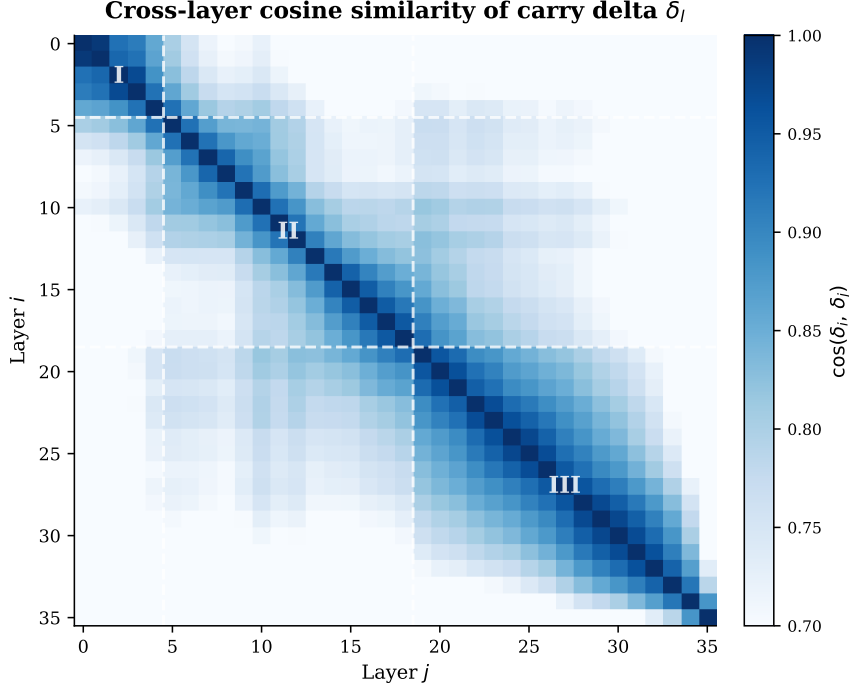


Figure 4.3: Cross-layer cosine similarity matrix $\cos(\delta_i, \delta_j)$ for all layer pairs. Dashed white lines mark the phase boundaries at layers 4–5 and 18–19. The block structure confirms that the carry direction is more similar within phases than across them, and that phases I and III share the least alignment, indicating a qualitative reorientation of the representation between early and late processing stages.

The matrix reveals a clear block-diagonal structure aligned with the three phases. Within each phase, cosine similarities are high (above 0.90 on the diagonal band), confirming that the carry direction is stable and approximately non-rotating within a processing stage. Across phases, similarities are lower, particularly between Phase I and Phase III where the off-diagonal blocks approach 0.70–0.75. This quantifies the degree to which the early and late carry representations, while pointing in the same broad direction, differ meaningfully in subspace orientation. The carry concept is not re-encoded from scratch at each transition but is continuously refined, with the phase boundaries marking the most significant directional shifts. This geometry is consistent with the inter-layer alignment ρ_l remaining above 0.88 everywhere while still producing substantial cumulative rotation across the full depth of the network.

4.7.3 Template Robustness and Output Preparation

The bottom panel of Figure 4.2 shows cross-template consistency throughout the network. All three template pairs remain above 0.97 through layer 24, confirming that the extracted carry direction is a genuine property of the model’s internal computation rather than an artefact of any particular surface form. The same direction emerges independently of whether the operands are presented in calculator syntax or natural language.

Beyond layer 24 a divergence emerges specifically between the natural-language template T2 and the two calculator-format templates T0 and T1 (which differ only in whitespace and stay aligned at 0.94 through layer 35). By layer 35, T1 vs T2 falls to 0.78. The divergence is driven not by a difference in carry computation itself but by how the late layers specialise the representation for the expected output format: T2 elicits a carry direction that begins to rotate toward token probabilities appropriate for natural-language continuation rather than a numeric calculator response.

This establishes a clean separation between two regimes within Phase III. In layers 19–24, the representation is still template-invariant and conceptually interpretable. From approximately layer 25 onward, it becomes output-specific. The transition at layer 25 thus marks the boundary between *concept encoding* and *output preparation* in the model’s arithmetic processing pipeline, a distinction that has practical implications for where interventions such as steering vectors are most likely to act on the underlying computation rather than on output formatting.

4.7.4 Feature-Space Decomposition

Figure 4.4 shows the top-15 transcoder features most aligned with the carry delta δ_l at each layer. Each point represents a feature f at layer l , with colour encoding the sign of $c_{l,f}$ (red positive, blue negative) and dot area proportional to $|c_{l,f}|^{1.5}$. The bottom strip repeats the carry delta norm for reference.

Across all 36 layers, the 540 selected feature–layer pairs draw from 539 distinct feature identifiers. Only one feature, F73141, appears in the top-15 at more than one layer (layers 21 and 28), a persistence rate below 0.2 per cent of the full feature vocabulary of approximately 160,000 components. This near-complete non-overlap is a strong mechanistic result: carry is not encoded through a small, reused set of monosemantic components, but through a layer-specific sparse basis in which each depth selects a largely disjoint subset of the transcoder dictionary.

The finding is consistent with the phase structure and cross-layer geometry described above. The block-diagonal similarity matrix in Figure 4.3 shows that within-phase directional alignment is high (above 0.90), predicting that features should be more stable within a phase than across phase transitions. The scatter confirms this qualitatively:

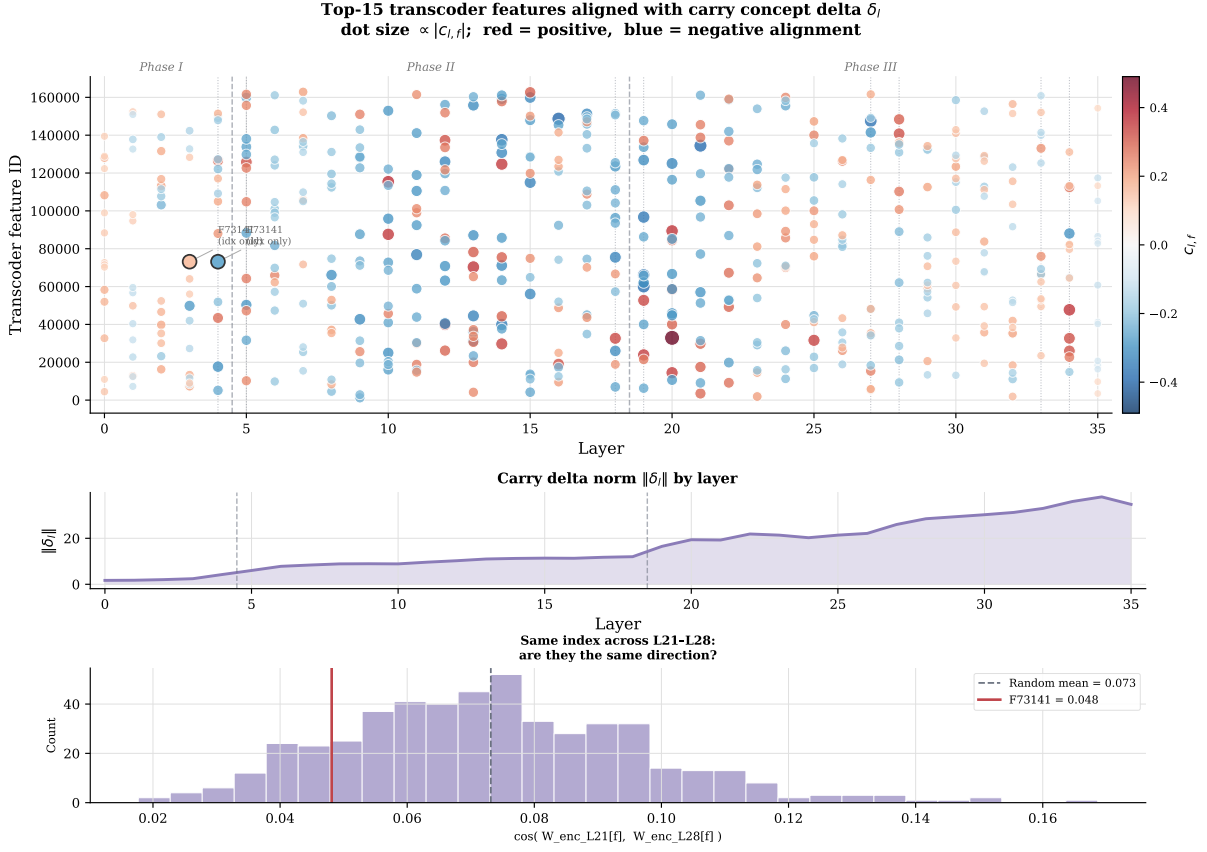


Figure 4.4: Top-15 transcoder features most aligned with the carry concept delta δ_l at each layer (equation 4.4). Dot area is proportional to $|c_{l,f}|^{1.5}$; red denotes positive alignment (feature activates more for carry), blue negative (activates more for no-carry). Phase boundaries are dashed vertical lines. Feature F73141 (outlined dots) is the only feature that recurs in the top-15 at more than one layer, appearing at layers 21 and 28 (annotations above and below the respective points). The bottom strip shows $\|\delta_l\|$ for context. Carry is encoded via a layer-specific sparse basis: the top-aligned features are almost entirely non-overlapping across layers, with no single monosemantic component persisting through the network.

visible banding in feature identity is apparent within each phase region, with the sharpest changeovers at the two transition boundaries.

The result also informs the scope of feature-level intervention. If carry were implemented through a small set of persistent features, a transcoder-based surgical edit targeting those features would be a natural next step. The layer-specific basis implies instead that any feature-level intervention would need to target a different subset at each depth. This reinforces the conclusion of Chapter ?? that computation-mode layer selection, rather than individual feature targeting, is the more tractable lever for influencing carry processing in a large pre-trained model.

4.8 Relation to Steering Effectiveness

The phase structure identified through contrastive activation analysis provides an independent, concept-level explanation for the layer-wise pattern of steering improvement reported in Chapter ?? . Figure 4.5 aligns the three relevant signals on a shared layer axis: the carry concept delta norm $\|\delta_l\|$, the steering vector norms $\|v_{k,l}\|$ for all four arithmetic primitives, and the per-layer improvement in full-answer accuracy produced by the FSM-routed steering intervention at the best amplitude $\alpha = 5.0$.

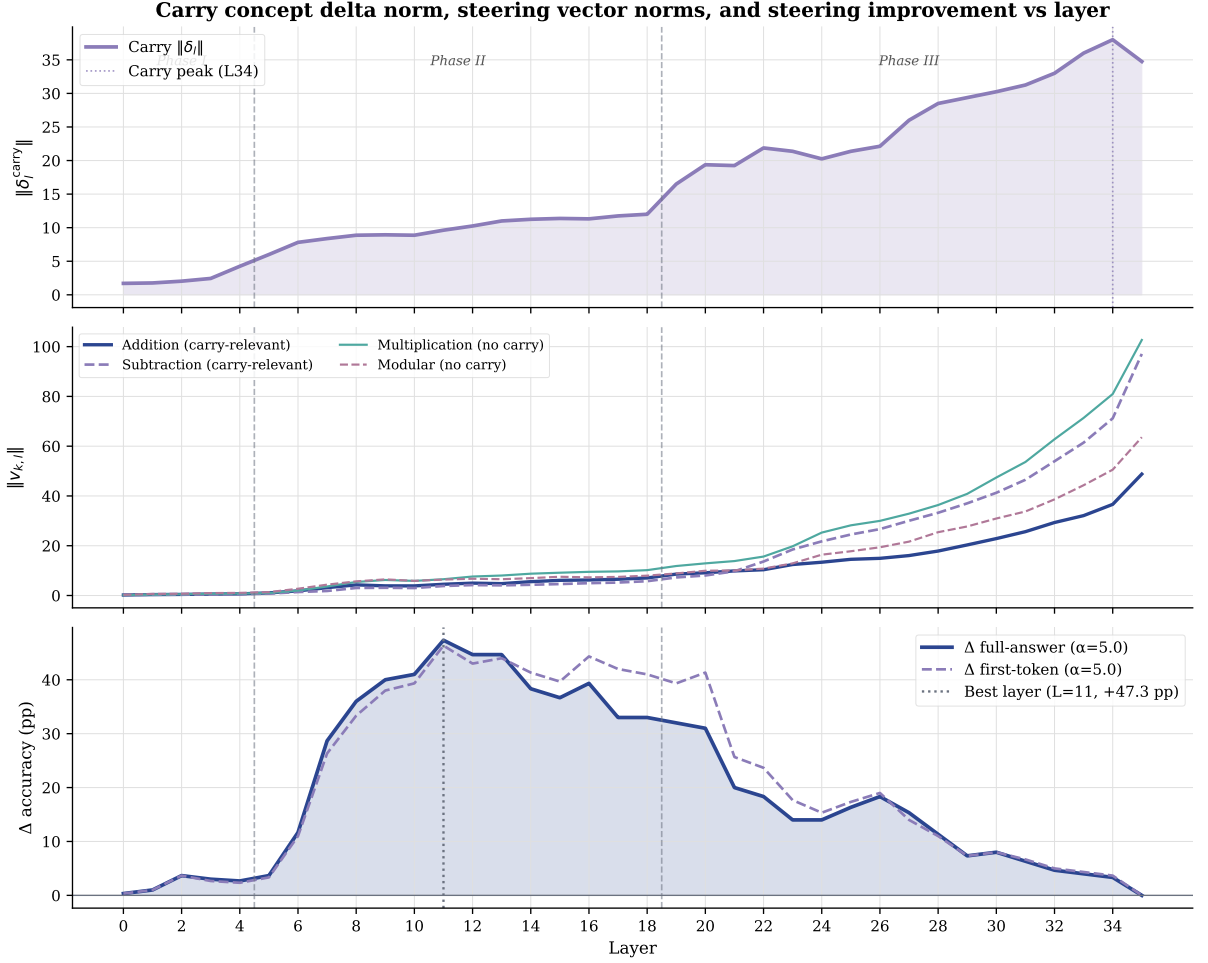


Figure 4.5: Three-panel comparison across all 36 layers. Top: carry concept delta norm $\|\delta_l\|$ (purple) with phase regions shaded and the peak at layer 34 marked. Middle: steering vector norms $\|v_{k,l}\|$ for addition and subtraction (carry-relevant, solid and dashed green) and for multiplication and modular (no carry event, red and orange). Bottom: layer-wise change in full-answer and first-token accuracy produced by local FSM steering at $\alpha = 5.0$, with the optimal injection layer $l^* = 11$ marked. Phase boundaries at layers 4–5 and 18–19 are shown as vertical dashed lines in all panels.

A naive comparison of the carry norm and the steering improvement curves yields no linear correlation (Pearson $r = -0.32$, Spearman $r = -0.12$, both non-significant), and the sign is informative. The carry norm grows monotonically to its peak at layer 34 precisely

as the steering improvement collapses toward zero, so a global linear relationship between the two signals would be mechanistically misleading. The carry direction becoming *larger* in Phase III does not make it a better target for intervention; as the template consistency analysis showed, Phase III is the regime in which the representation is being projected toward output-logit space rather than maintained as a computation-level encoding.

The meaningful correspondence is structural rather than scalar. The three-phase boundary identified in the carry analysis predicts the effective steering window with high fidelity. In Phase I (layers 0–4) the carry representation has not yet crystallised, and the steering improvement is negligible across all amplitudes. The 4–5 transition, the first point at which carry information becomes linearly stable, coincides with the onset of meaningful steering gain. Phase II (layers 5–18) is the regime of stable, template-invariant carry encoding identified in Section 4.5: it is also the regime in which steering improvement rises sharply, peaking at layer 11 with a full-answer accuracy gain of +47.3 percentage points over the unsteered baseline. The 18–19 transition, the entry into Phase III, coincides with the onset of steering degradation: improvement falls continuously thereafter, and by layers 30–35 the intervention is marginally harmful.

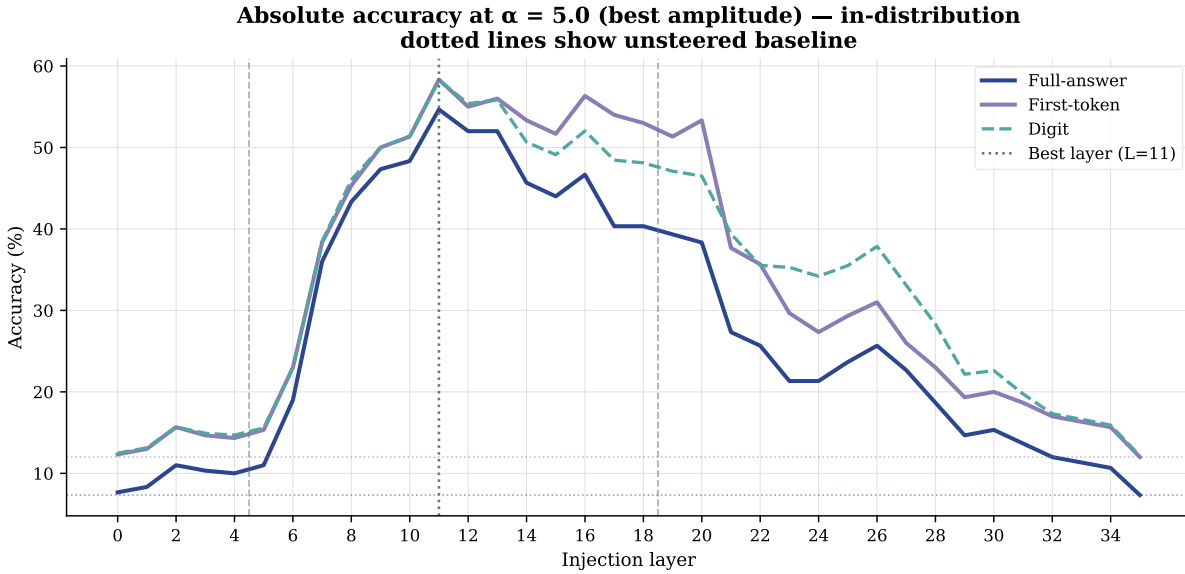


Figure 4.6: Absolute accuracy at the best amplitude $\alpha = 5.0$ as a function of injection layer, for full-answer (green), first-token (blue), and digit (dashed orange) metrics. Dotted horizontal lines show the unsteered baseline for each metric. The bell-shaped profile peaks at layer 11 and falls to near-baseline levels by layer 30, matching the Phase II window of stable carry encoding.

Figure 4.6 makes the bell-shaped layer profile of steering improvement explicit. The peak at layer 11 falls squarely within Phase II, and the return to baseline in Phase III is consistent with the output-preparation hypothesis: injecting a computation-mode steering vector into layers that are already projecting toward output tokens disrupts the formatting

of the response without reinforcing the underlying arithmetic operation.

A further observation concerns the steering vector norms shown in the middle panel of Figure 4.5. The norms for the four primitives are nearly indistinguishable and all grow monotonically, driven by residual-stream scale accumulation rather than by any primitive-specific structure. In particular, addition and subtraction, both subject to carry events, track multiplication and modular, which involve no carry, throughout the network. This confirms that the sv norm alone is an unreliable predictor of steering effectiveness: the relevant factor is the geometric orientation of the extracted direction relative to the computation being performed at that depth, not its Euclidean magnitude.

Taken together, the carry localisation analysis and the steering evaluation are mutually consistent and mutually explanatory. The concept-level analysis predicts which layers are computation-mode layers and which are output-preparation layers; the intervention experiment confirms that only computation-mode layers admit effective steering. The two results are produced by entirely independent pipelines, contrastive activation averaging on matched carry pairs and confidence-bucketed extraction followed by causal injection, and their agreement strengthens the mechanistic interpretation of both.

Bibliography

- [1] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Shan Carter, Tom Henighan, Adam Stevenson, Tom Conerly, Danny Hernandez, Adam Jermyn, Callum McDougall, et al. On the biology of a large language model. 2025. URL <https://transformer-circuits.pub/2025/biology-of-a-lm/index.html>. Anthropic Transformer Circuits Thread.
- [2] Tiberiu Musat. Clustering and alignment: Understanding the training dynamics in modular addition, 2024.
- [3] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2301.05217>.
- [4] Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models, 2023.
- [5] Philip Quirke and Fazl Barez. Understanding addition in transformers. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2310.13121>.
- [6] Philip Quirke, Clement Neo, and Fazl Barez. Understanding addition and subtraction in transformers, 2025.
- [7] Nick Rimskey, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition, 2023.
- [8] Ziqian Zhong, Ziming Liu, Max Tegmark, and Jacob Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks. In *Advances in Neural Information Processing Systems*, 2023. doi: 10.48550/arXiv.2306.17844.
- [9] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Hubert Li, Andeng Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Long Long, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo,

Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to ai transparency, 2023.

Glossary

Attribution graph A directed acyclic graph whose nodes represent model components (token embeddings, SAE or CLT features, attention heads, output logits) and whose edges represent gradient-weighted information flow between them. Introduced as a methodology by Lindsey et al. [1].

Circuit A subgraph of the model’s computational graph comprising a small set of components (attention heads, MLP layers, or individual features) that together implement a specific behaviour. The term is used in mechanistic interpretability to denote minimal, interpretable sub-computations.

CLT — Cross-Layer Transcoder A sparse dictionary-learning module, analogous to a sparse autoencoder, that maps MLP inputs at one layer to residual-stream contributions, potentially spanning across layers.

FFN — Feed-Forward Network Synonym for MLP in the transformer literature.

Grokking The phenomenon whereby a neural network trained on a small dataset first memorises the training examples (near-perfect training accuracy, poor generalisation) and then, after many additional gradient steps, abruptly transitions to a generalising solution. Mechanistically characterised for modular arithmetic by Nanda et al. [3].

LLM — Large Language Model A neural language model with a parameter count large enough that emergent capabilities arise from scale alone, typically trained on broad natural-language corpora via next-token prediction.

MLP — Multilayer Perceptron In the transformer architecture, the feed-forward sub-layer of each block, consisting of two learned linear projections with a pointwise nonlinear activation in between. Despite the name, transformer MLPs typically have only a single hidden layer.

RMSNorm — Root Mean Square Layer Normalisation A variant of layer normalisation that rescales activations by their root mean square without subtracting the mean. Used in Qwen and other modern architectures in place of standard LayerNorm.

SAE — Sparse Autoencoder A dictionary-learning module trained to decompose neural network activations into a sparse linear combination of learned feature directions. Used in mechanistic interpretability to identify human-interpretable features inside a model.

Representation engineering A methodology for extracting and analysing concept directions in the activation space of a neural network by computing the mean difference in hidden-state vectors between semantically contrasting inputs. Proposed by Zou et al. [9] as a top-down approach to model transparency, complementary to the bottom-up circuit-based programme.

Sharpness index A scalar summary of how tightly a concept delta’s norm is concentrated around its peak layer, defined as the fraction of total norm mass lying within a window of width three centred at the layer of maximum norm. Values close to one indicate a localised representation; values close to $3/L$ indicate a representation that is uniformly distributed across the network’s L layers.